

ОТЧЕТ ПО HW4

Фролов Иван Григорьевич

БПИ-235

Цель работы

написать файловое хранилище MyDrive с использованием Java/Netty Framework и протокола TCP.

Хотим сделать клиент-серверное приложение, где:

- Клиент синхронизирует директорию с сервером
- Сервер хранит файлы каждого пользователя в отдельной директории
- Используется собственный протокол TCP для обмена сообщениями

Язык программирования и Framework: Java 11 / Netty Framework 4.1

Структура проекта

```
mydrive/
├── protocol/
│   ├── Message.java           (базовый интерфейс)
│   ├── MessageType.java      (константы типов сообщений)
│   └── MessageEncoder.java    (сериализует сообщение в
байты)
│   ├── MessageDecoder.java    (десериализует)
│   ├── ClientIdMessage.java   (ID клиента)
│   └── FileListMessage.java   (список файлов с
контрольными суммами)
│   ├── FileResponseMessage.java (список отсутствующих
файлов)
│   └── FileChunkMessage.java  (данные файла)
├── server/
│   ├── MyDriveServer.java     (стартуем сервер)
│   └── ServerHandler.java     (обработчик подключений)
├── client/
│   ├── MyDriveClient.java     (стартуем клиента)
│   └── ClientHandler.java     (обработчик ответов)
└── util/
    └── FileUtils.java         (вспомогательные функции)
```

Протокол коммуникации

Основан на слайде №14 лекции №9:

```
Сообщение = [Type (1 byte)] [Length (2 bytes)] [Data (variable)]
```

Типы сообщений:

1. CLIENT_ID – отправляет клиент при подключении
2. FILE_LIST – список файлов клиента с контрольными суммами
3. FILE_RESPONSE – сервер отвечает, какие файлы нужны
4. FILE_CHUNK – передача содержимого файла

Поток взаимодействия

1. Клиент подключается → отправляет CLIENT_ID
2. Клиент сканирует локальную директорию и отправляет FILE_LIST
3. Сервер проверяет файлы → отправляет FILE_RESPONSE
4. Клиент отправляет отсутствующие файлы через FILE_CHUNK
5. Сервер сохраняет файлы в /storage/[client_id]/[filename]

Вот пример запуска

Компилируем и очищаем облако

```
mvn clean package -DskipTests 2>&1 && rm -rf /tmp/mydrive
```

Запуск сервера

```
java -jar target/mydrive-server.jar 9999 /tmp/mydrive
```

Запуск клиента

```
./run_client.sh config.properties
```

Создать тестовые файлы

```
./create_test_files.sh
```

Все файлы примерно по 100 мб

```
Sent client ID: user1
Scanned 10 files
Sent file list with 10 files
Received missing files list: [file_9.bin, file_4.bin, file_3.bin, file_2.bin, file_10.bin, file_6.
bin, file_1.bin, file_5.bin, file_8.bin, file_7.bin]
Sent file: file_9.bin (104857600 bytes)
Sent file: file_4.bin (104857600 bytes)
Sent file: file_3.bin (104857600 bytes)
Sent file: file_2.bin (104857600 bytes)
Sent file: file_10.bin (104857600 bytes)
Sent file: file_6.bin (104857600 bytes)
Sent file: file_1.bin (104857600 bytes)
Sent file: file_5.bin (104857600 bytes)
Sent file: file_8.bin (104857600 bytes)
Sent file: file_7.bin (104857600 bytes)
Sync completed in 7446 ms
```

Клиент коннектится к серверу, отправляет ему информацию о тех файлах, которые у него есть локально (то есть в папке ./test_files)

Далее сервер возвращает клиенту список имен тех файлов, которые он у себя не нашел в облаке (наше условное облако - это директория ~/tmp/mydrive)

```
Server started on port 9999, storage: /tmp/mydrive
Client connected: user1
Sent file response for client: user1, missing: 10
```

Соответственно, дальше клиент отправляет серверу только те файлы, которых у него еще нет

После этого можно посмотреть, что сервер получил все файлы в свое облако

```
admin@MacBook-Pro Computer Networks % ls -a /tmp/mydrive/user1
./          file_1.bin  file_2.bin  file_4.bin  file_6.bin  file_8.bin
../         file_10.bin file_3.bin  file_5.bin  file_7.bin  file_9.bin
```

Можно посмотреть в Wireshark, что TCP пакеты действительно перемещаются между клиентом и сервером

Capturing from L

tcp.port == 9999

No.	Time	Source	Destination	Protocol	Length	Info
1014	109.972608	127.0.0.1	127.0.0.1	TCP	68	57180 → 9999 [SYN
1015	109.972653	127.0.0.1	127.0.0.1	TCP	68	9999 → 57180 [SYN
1016	109.972659	127.0.0.1	127.0.0.1	TCP	56	57180 → 9999 [ACK
1017	109.972665	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Updat
1018	109.977406	127.0.0.1	127.0.0.1	TCP	64	57180 → 9999 [PSH
1019	109.977422	127.0.0.1	127.0.0.1	TCP	56	9999 → 57180 [ACK
1020	110.284258	127.0.0.1	127.0.0.1	TCP	59	57180 → 9999 [PSH
1021	110.284293	127.0.0.1	127.0.0.1	TCP	56	9999 → 57180 [ACK
1036	117.776703	127.0.0.1	127.0.0.1	TCP	56	57180 → 9999 [FIN
1037	117.776780	127.0.0.1	127.0.0.1	TCP	56	9999 → 57180 [ACK
1038	117.778421	127.0.0.1	127.0.0.1	TCP	56	9999 → 57180 [FIN
1039	117.778482	127.0.0.1	127.0.0.1	TCP	56	57180 → 9999 [ACK
1701	168.230338	127.0.0.1	127.0.0.1	TCP	68	57225 → 9999 [SYN
1702	168.230424	127.0.0.1	127.0.0.1	TCP	68	9999 → 57225 [SYN
1703	168.230440	127.0.0.1	127.0.0.1	TCP	56	57225 → 9999 [ACK
1704	168.230452	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Updat
1705	168.234979	127.0.0.1	127.0.0.1	TCP	64	57225 → 9999 [PSH
1706	168.234990	127.0.0.1	127.0.0.1	TCP	56	9999 → 57225 [ACK
1721	168.540996	127.0.0.1	127.0.0.1	TCP	59	57225 → 9999 [PSH

> Frame 1014: Packet, 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface lo

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> Transmission Control Protocol, Src Port: 57180, Dst Port: 9999, Seq: 0, Len: 0

Source Port: 57180

Destination Port: 9999

[Stream index: 39]

[Stream Packet Number: 1]

> [Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 0]

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 2048880577

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 0

Acknowledgment number (raw): 0

1011 = Header Length: 44 bytes (11)

> Flags: 0x002 (SYN)

Window: 65535

[Calculated window size: 65535]

Checksum: 0xfe34 [unverified]

Флаг SYN - синхронизация

PSH/ACK - пуш и потом acknowledge

TCP Window update - технический флаг, сообщить о доступном размере буфера

Ключевые фрагменты кода

1. Message Decoder

```
public class MessageDecoder extends ByteToMessageDecoder {
    @Override
    protected void decode(ChannelHandlerContext ctx, ByteBuf in,
        List<Object> out) {
        if (in.readableBytes() < 4) return;

        in.markReaderIndex();
        int messageType = in.readByte();

        if (messageType == MessageType.CLIENT_ID) {
            int idLength = in.readShort();
            if (in.readableBytes() < idLength) {
                in.resetReaderIndex();
                return;
            }
            byte[] idBytes = new byte[idLength];
            in.readBytes(idBytes);
            out.add(new ClientIdMessage(new String(idBytes)));
        }
        // ... другие типы сообщений
    }
}
```

Суть: используем ByteToMessageDecoder для обработки потока байтов из TCP соединения. Проверяем наличие данных перед чтением без блокировки, потом десериализуем в объекты.

2. Message Encoder

```
public class MessageEncoder extends MessageToByteEncoder<Message>
{
    @Override
    protected void encode(ChannelHandlerContext ctx, Message msg,
        ByteBuf out) {
        if (msg instanceof ClientIdMessage) {
            ClientIdMessage m = (ClientIdMessage) msg;
            out.writeByte(MessageType.CLIENT_ID);
            out.writeShort(m.getClientId().length());
            out.writeBytes(m.getClientId().getBytes());
        }
    }
}
```

```

        // ... другие типы
    }
}

```

Суть: преобразуем объекты Java в последовательность байтов для отправки по TCP.

3. Server Handler (обработка клиентов)

```

public class ServerHandler extends
SimpleChannelInboundHandler<Message> {
    private String clientId;
    private String storageDir;
    private Map<String, byte[]> serverFiles = new HashMap<>();

    @Override
    protected void channelRead0(ChannelHandlerContext ctx,
Message msg) {
        if (msg instanceof ClientIdMessage) {
            clientId = ((ClientIdMessage) msg).getClientId();
            String clientDir = storageDir + File.separator +
clientId;
            FileUtils.getOrCreateDirectory(clientDir);
            loadServerFiles(clientDir);

        } else if (msg instanceof FileListMessage) {
            FileListMessage m = (FileListMessage) msg;
            FileResponseMessage response = new
FileResponseMessage();

            for (FileListMessage.FileInfo file : m.GetFiles()) {
                byte[] serverChecksum =
serverFiles.get(file.fileName);
                if (serverChecksum == null ||
!Arrays.equals(serverChecksum, file.checksum)) {
                    response.addMissingFile(file.fileName);
                }
            }
            ctx.writeAndFlush(response);

        } else if (msg instanceof FileChunkMessage) {
            FileChunkMessage m = (FileChunkMessage) msg;
            // Сохраняем файл в clientDir
            // Вычисляем контрольную сумму после полного
получения
        }
    }
}

```

Суть: для каждого подключения Netty создает отдельный handler. Каждый клиент управляется независимо в одном потоке события.

Поддерживается неограниченное число клиентов (event loop group).

4. Клиентская логика

```
public void sync() throws Exception {
    Channel channel = connect();

    sendClientId(channel);
    Thread.sleep(500);

    Map<String, FileInfo> localFiles = scanLocalDirectory();
    sendFileList(channel, localFiles);
    Thread.sleep(500);

    List<String> missingFiles = waitForResponse(channel);
    sendMissingFiles(channel, localFiles, missingFiles);

    channel.close().sync();
}
```

Суть: коннектится к серверу, сообщает свое id, отправляет список файлов, получает список MissingFiles, отправляет недостающие файлы

Сериализация сообщений

Каждый тип сообщения имеет фиксированный формат:

```
ClientIdMessage:
    [1 byte: type=1][2 bytes: id_length][id_length bytes: id]

FileListMessage:
    [1 byte: type=2][2 bytes: file_count]
    [для каждого файла]:
        [2 bytes: name_length][name_length bytes: name]
        [8 bytes: file_size]
        [16 bytes: checksum]

FileChunkMessage:
    [1 byte: type=4][2 bytes: name_length][name_length bytes: name]
    [8 bytes: total_file_size][N bytes: chunk_data]
```

Тут дальше будем реализовывать parallel и DMA

Общая концепция:

- сервер пассивный, он просто принимает и обрабатывает, что ему придет. Уже поддерживает параллельные клиенты из коробки.

- Клиент может выбирать стратегию отправки (последовательно / параллельно) и метод отправки (с копированием или DMA)

Параллельная отправка нескольких файлов

Для этого написан отдельный клиент `MyDriveClientParallel.java`

Он создает пул потоков и до `max.connections` одновременных подключений, каждое подключение отправляет отдельный файл

Сервер уже умеет работать с многопоточностью

Сценарий №1: один клиент с несколькими потоками

Терминал 1: Очищаем облако и перезапускаем сервер

```
rm -rf /tmp/mydrive
java -jar target/mydrive-server.jar 9999 /tmp/mydrive
```

Терминал 3: Запускаем параллельного клиента

```
java -cp target/mydrive-server.jar
mydrive.client.MyDriveClientParallel config.properties
```

```
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
Server started on port 9999, storage: /tmp/mydrive
Client connected: user1
Sent file response for client: user1, missing: 10
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
Client connected: user1
```

Более наглядно можно видеть в WireShark, что создается несколько подключений. То есть у нас несколько портов для клиента.

tcp.port == 9999						
No.	Time	Source	Destination	Protocol	Length	Info
416	20.582418	127.0.0.1	127.0.0.1	TCP	68	59038 →
417	20.582511	127.0.0.1	127.0.0.1	TCP	68	9999 → 5
418	20.582527	127.0.0.1	127.0.0.1	TCP	56	59038 →
419	20.582542	127.0.0.1	127.0.0.1	TCP	56	[TCP Win
420	20.587033	127.0.0.1	127.0.0.1	TCP	64	59038 →
421	20.587050	127.0.0.1	127.0.0.1	TCP	56	9999 → 5
428	22.437777	127.0.0.1	127.0.0.1	TCP	420	59038 →
429	22.437814	127.0.0.1	127.0.0.1	TCP	56	9999 → 5
430	22.438652	127.0.0.1	127.0.0.1	TCP	180	9999 → 5
431	22.438672	127.0.0.1	127.0.0.1	TCP	56	59038 →
450	22.949103	127.0.0.1	127.0.0.1	TCP	68	59043 →
451	22.949139	127.0.0.1	127.0.0.1	TCP	68	59044 →
452	22.949169	127.0.0.1	127.0.0.1	TCP	68	9999 → 5
453	22.949195	127.0.0.1	127.0.0.1	TCP	68	9999 → 5
454	22.949204	127.0.0.1	127.0.0.1	TCP	56	59043 →
455	22.949208	127.0.0.1	127.0.0.1	TCP	56	59044 →
456	22.949226	127.0.0.1	127.0.0.1	TCP	56	[TCP Win
457	22.949229	127.0.0.1	127.0.0.1	TCP	56	[TCP Win
458	22.949380	127.0.0.1	127.0.0.1	TCP	68	59045 →

> Frame 430: Packet, 180 bytes on wire (1440 bits), 180 bytes captured (1440 bits) on ir
 > Null/Loopback
 > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 √ Transmission Control Protocol, Src Port: 9999, Dst Port: 59038, Seq: 1, Ack: 373, Len:

Source Port: 9999
 Destination Port: 59038
 [Stream index: 10]
 [Stream Packet Number: 9]
 > [Conversation completeness: Complete, WITH_DATA (31)]
 [TCP Segment Len: 124]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 3423413389
 [Next Sequence Number: 125 (relative sequence number)]
 Acknowledgment Number: 373 (relative ack number)
 Acknowledgment number (raw): 2233178830

tcp.port == 9999						
No.	Time	Source	Destination	Protocol	Length	Info
428	22.437777	127.0.0.1	127.0.0.1	TCP	420	59038 → 9999 [
429	22.437814	127.0.0.1	127.0.0.1	TCP	56	9999 → 59038 [
430	22.438652	127.0.0.1	127.0.0.1	TCP	180	9999 → 59038 [
431	22.438672	127.0.0.1	127.0.0.1	TCP	56	59038 → 9999 [
450	22.949103	127.0.0.1	127.0.0.1	TCP	68	59043 → 9999 [
451	22.949139	127.0.0.1	127.0.0.1	TCP	68	59044 → 9999 [
452	22.949169	127.0.0.1	127.0.0.1	TCP	68	9999 → 59043 [
453	22.949195	127.0.0.1	127.0.0.1	TCP	68	9999 → 59044 [
454	22.949204	127.0.0.1	127.0.0.1	TCP	56	59043 → 9999 [
455	22.949208	127.0.0.1	127.0.0.1	TCP	56	59044 → 9999 [
456	22.949226	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Up
457	22.949229	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Up
458	22.949380	127.0.0.1	127.0.0.1	TCP	68	59045 → 9999 [
459	22.949416	127.0.0.1	127.0.0.1	TCP	68	9999 → 59045 [
460	22.949424	127.0.0.1	127.0.0.1	TCP	56	59045 → 9999 [
461	22.949429	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Up
462	22.949491	127.0.0.1	127.0.0.1	TCP	68	59046 → 9999 [
463	22.949518	127.0.0.1	127.0.0.1	TCP	68	9999 → 59046 [
464	22.949525	127.0.0.1	127.0.0.1	TCP	56	59046 → 9999 [
> Frame 454: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface lo > Null/Loopback > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 > Transmission Control Protocol, Src Port: 59043, Dst Port: 9999, Seq: 1, Ack: 1, Len: 0 Source Port: 59043 Destination Port: 9999 [Stream index: 12] [Stream Packet Number: 3] > [Conversation completeness: Complete, WITH_DATA (31)] [TCP Segment Len: 0] Sequence Number: 1 (relative sequence number) Sequence Number (raw): 1344403497 [Next Sequence Number: 1 (relative sequence number)]						

tcp.port == 9999							
No.	Time	Source	Destination	Protocol	Length	Info	
428	22.437777	127.0.0.1	127.0.0.1	TCP	420	59038 → 9999	[F
429	22.437814	127.0.0.1	127.0.0.1	TCP	56	9999 → 59038	[A
430	22.438652	127.0.0.1	127.0.0.1	TCP	180	9999 → 59038	[F
431	22.438672	127.0.0.1	127.0.0.1	TCP	56	59038 → 9999	[A
450	22.949103	127.0.0.1	127.0.0.1	TCP	68	59043 → 9999	[S
451	22.949139	127.0.0.1	127.0.0.1	TCP	68	59044 → 9999	[S
452	22.949169	127.0.0.1	127.0.0.1	TCP	68	9999 → 59043	[S
453	22.949195	127.0.0.1	127.0.0.1	TCP	68	9999 → 59044	[S
454	22.949204	127.0.0.1	127.0.0.1	TCP	56	59043 → 9999	[A
455	22.949208	127.0.0.1	127.0.0.1	TCP	56	59044 → 9999	[A
456	22.949226	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Upd	
457	22.949229	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Upd	
458	22.949380	127.0.0.1	127.0.0.1	TCP	68	59045 → 9999	[S
459	22.949416	127.0.0.1	127.0.0.1	TCP	68	9999 → 59045	[S
460	22.949424	127.0.0.1	127.0.0.1	TCP	56	59045 → 9999	[A
461	22.949429	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Upd	
462	22.949491	127.0.0.1	127.0.0.1	TCP	68	59046 → 9999	[S
463	22.949518	127.0.0.1	127.0.0.1	TCP	68	9999 → 59046	[S
464	22.949525	127.0.0.1	127.0.0.1	TCP	56	59046 → 9999	[A

> Frame 455: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface lo
 > Null/Loopback
 > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 > Transmission Control Protocol, Src Port: 59044, Dst Port: 9999, Seq: 1, Ack: 1, Len: 0
 Source Port: 59044
 Destination Port: 9999
 [Stream index: 13]
 [Stream Packet Number: 3]
 > [Conversation completeness: Complete, WITH_DATA (31)]
 [TCP Segment Len: 0]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 1284256541
 [Next Sequence Number: 1 (relative sequence number)]

Тут мы видим порты 59038, 59043, 59044

Сценарий №2: несколько клиентов

Запустить скрипт тестирования: он запустит и сервер, и клиентов

```
chmod +x test_parallel.sh
./test_parallel.sh 2 true 9998
```

Параметры:

- 3 — число клиентов
- true — использовать параллельный режим (или false для последовательного)
- 9998 - порт

Можно посмотреть на tcp dump (результаты в файле tcp_analysis.txt)

Что можно сказать:

1. MSS = 16344 байт. Window Start = 65535 → уменьшается до 4332 (Flow Control)
2. Медленный старт в первые 400ms
3. Пакеты растут: 8B → 69B → 16KB → 65KB

4. Congestion нет (у нас локальное соединение)
 5. Flow Control работает, 2 независимых соединения
-

Использование DMA

Для этого есть отдельный клиент `MyDriveClientDMA.java`

Там изменение `sendFile` на использование `channel.writeAndFlush(new DefaultFileRegion(file, 0, file.length()))` и предварительная отправка заголовка с именем/размером

Запустить все для проверки можно скриптом

`test_parallel.sh`

Что получается:

```
admin@MacBook-Pro hw4 % ./test_dma.sh
Creating test files (100MB each)...
Done.

Starting server on port 9997...
=== Test 1: WITHOUT DMA ===
=== Test 2: WITH DMA ===

=== Results ===
Without DMA: 0m6.856s
With DMA:    0m6.044s

=== Files on Server ===

=== Logs ===
Client without DMA:
Sent file (non-DMA): file_2.bin (104857600 bytes)
  Time: 343 ms, DMA=false
Sent file (non-DMA): file_1.bin (104857600 bytes)
  Time: 220 ms, DMA=false
Sync completed in 5812 ms (DMA=false)

Client with DMA:
Sent file (DMA): file_2.bin (104857600 bytes)
  Time: 417 ms, DMA=true
Sent file (DMA): file_1.bin (104857600 bytes)
  Time: 342 ms, DMA=true
Sync completed in 5999 ms (DMA=true)
```

Результаты DMA теста

Общие времена:

- Без DMA: 6.856s
- С DMA: 6.044s

- Ускорение: 0.8s (11.8% быстрее)

Пропускная способность:

- Без DMA: $200\text{MB} \div 6.856\text{s} = 29.2 \text{ MB/s}$
- С DMA: $200\text{MB} \div 6.044\text{s} = 33.1 \text{ MB/s}$
- Прирост: +13%

Эффект тут получился небольшой

Фух, вроде все